

PRACTCAL Week 4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
GNU nano 7.2                                wait_compare.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2, child3;
    int status;

    printf("Parent Process: PID = %d\n\n", getpid());

    // Create Child 1
    child1 = fork();

    if (child1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 completed.\n");
        exit(1);
    }

    // Create Child 2
    child2 = fork();

    if (child2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed.\n");
        exit(2);
    }

    // Create Child 3
    child3 = fork();

    if (child3 == 0) {
        printf("Child 3: PID = %d\n", getpid());
        sleep(2);
        printf("Child 3 completed.\n");
        exit(3);
    }

    // Parent uses wait()
    printf("\nParent using wait():\n");

    pid_t finished = wait(&status);

    printf("wait() collected child PID = %d\n", finished);

    if (WIFEXITED(status))
        printf("Exit status = %d\n", WEXITSTATUS(status));

    // Parent uses waitpid()
    printf("\nParent using waitpid() for Child 1:\n");

    waitpid(child1, &status, 0);

    printf("waitpid() collected Child 1 PID = %d\n", child1);

    if (WIFEXITED(status))
        printf("Exit status = %d\n", WEXITSTATUS(status));

    // Collect remaining children
    waitpid(child3, &status, 0);
    printf("Remaining Child 3 completed.\n");

    waitpid(child2, &status, 0);
    printf("Remaining Child 2 completed.\n");

    printf("\nParent process completed.\n");

    return 0;
}
```

```

krishnasree08@krishnasree08:~$ ~$ nano wait_compare.c
krishnasree08@krishnasree08:~$ ~$ gcc wait_compare.c -o wait_compare
krishnasree08@krishnasree08:~$ ~$ ./wait_compare
Parent Process: PID = 1996

Child 1: PID = 1997

Child 2: PID = 1998
Parent using wait():
Child 3: PID = 1999
Child 2 completed.
wait() collected child PID = 1998
Exit status = 2

Parent using waitpid() for Child 1:
Child 3 completed.
Child 1 completed.
waitpid() collected Child 1 PID = 1997
Exit status = 1
Remaining Child 3 completed.
Remaining Child 2 completed.

Parent process completed.

```

ALGORITHM: Parent Creates Multiple Children and Synchronizes using waitpid()	
1.	START
2.	Declare an array child[NUM_CHILDREN] to store child PIDs and an integer status.
3.	Print a message indicating the parent process has started along with its PID.
4.	For $i \leftarrow 0$ to NUM_CHILDREN - 1 do 4.1 Call fork() and store the return value in child[i]. 4.2 If fork() < 0 then 4.3 Print error message and terminate the program. 4.4 Else if fork() == 0 then // Child process 4.4.1 Print message: "Child $i + 1$ started" along with child PID. 4.4.2 Sleep for $(i + 1) \times 2$ seconds (to create different completion times). 4.4.3 Print message: "Child $i + 1$ completed". 4.4.4 Exit the child process with exit status $(i + 1)$.
5.	End For
6.	Print message: Parent waiting for children using waitpid().
7.	For $i \leftarrow 0$ to NUM_CHILDREN - 1 do
8.	Call waitpid(child[i], &status, 0) and store the return value in pid.
9.	If waitpid() == -1 then
10.	Print error message and terminate the program.
11.	Else
12.	If WIFEXITED(status) is true then
13.	Print: Child with PID pid terminated with exit status WEXITSTATUS(status).
14.	End If
15.	End If
16.	End For
17.	Print message: All child processes completed. Parent exiting. STOP

Observation

1. The parent successfully creates **multiple child processes** using fork().
2. wait() collects **any child that terminates first**.
3. waitpid() allows the parent to **wait for a specific child using its PID**.
4. Both functions synchronize the parent and prevent child processes from remaining as zombies after they are collected.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
GNU nano 7.2 zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }

    else {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent is sleeping...\n");
        sleep(30);

        printf("Parent completed.\n");
    }

    return 0;
}
```

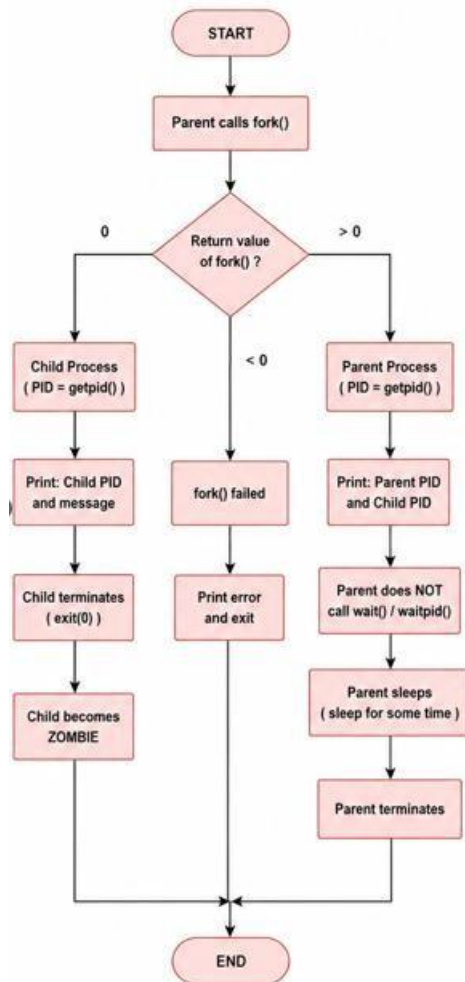
```
krishnasree08@krishnasree08:~$ nano zombie.c
krishnasree08@krishnasree08:~$ gcc zombie.c -o zombie
krishnasree08@krishnasree08:~$ ./zombie
```

```
Parent Process
Parent PID = 2029
Child PID = 2030
Parent is sleeping...
Child Process
Child PID = 2030
Child is terminating...
```

```
krishnasree08@krishnasree08:~$ ps -o pid,ppid,state,stat,cmd | grep 2156
  2159    2108 S S+  grep --color=auto 2156
krishnasree08@krishnasree08:~$ grep "^State" /proc/2156/status
State:  Z (zombie)
krishnasree08@krishnasree08:~$
```

Flow Chart:

FLOWCHART 1: CREATE ZOMBIE PROCESS
(Parent does NOT call waitpid())



During the time parent is sleeping,
child is shown in process table as Z (zombie / <defunct>)

```

GNU nano 7.2                                zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }

    else {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child collected successfully.\n");
        printf("No zombie process remains.\n");
    }

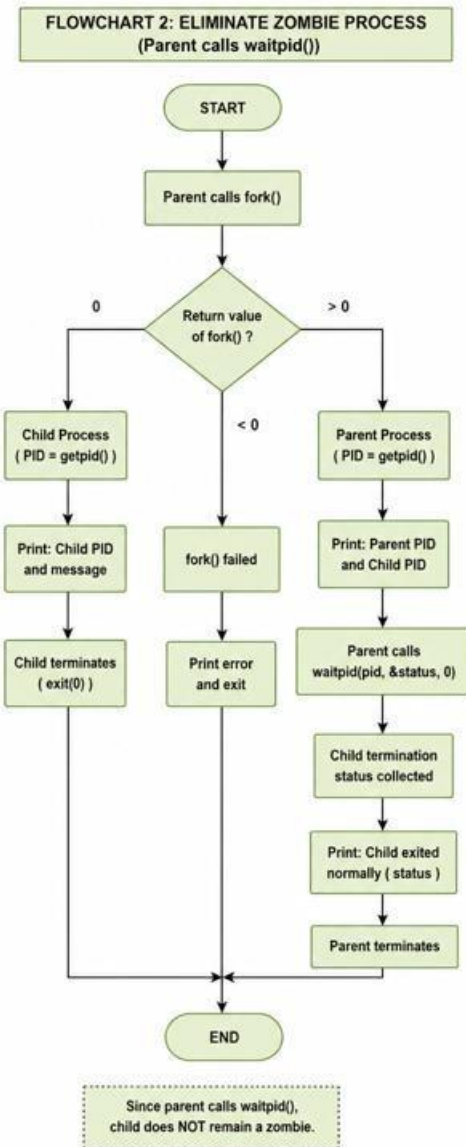
    return 0;
}

```

```

krishnasree08@krishnasree08:~$ nano zombie.c
krishnasree08@krishnasree08:~$ gcc zombie.c -o zombie
krishnasree08@krishnasree08:~$ ./zombie
Parent Process
Parent PID = 2178
Child PID = 2179
Parent waiting for child...
Child Process
Child PID = 2179
Child is terminating...
Child collected successfully.
No zombie process remains.
krishnasree08@krishnasree08:~$ ps -o pid,ppid,state,stat,cmd | grep zombie
 2181    1916 S S+  grep --color=auto zombie
krishnasree08@krishnasree08:~$

```



Final Observation

The child process initially became a zombie because the parent did not collect its exit status. After adding wait(), the parent waits for the child and collects its termination status, thereby eliminating the zombie process.